

End-to-End Playing Card Classification using a Feedforward and Sequential Convolutional Neural Network.

Cole Current, Aashish Harishchandre, Kyle Rockwell, Saatvik Palli

Abstract

Automatic identification of playing cards represents a unique challenge in computer vision. Between the wide variety of card designs and often heavy wear on the cards, it can be difficult to classify playing cards accurately, and while previous research has found success rates of up to 91.5% (Chen et al.), we hope to create an even more reliable and powerful model. We plan on primarily using the dataset from Chen et al., with 2981 labeled images of cards from various angles and distances; in addition, we have a labeled dataset from paperswithcode.com that contains isolated scans of 8029 playing cards. The simple model we used was a Feedforward Neural Network that the RReLU and ReLU activation functions. The more complex model we used was a Sequential Convolutional Neural Network. We used PyTorch to transform each image in the dataset into tensors to train each model. We finished with a final accuracy of 58.2% for the Feedforward Neural Network and 64.0% for the Sequential Convolutional Neural Network.

1 Introduction

Our team of four consists of Cole Current (currentc) a 4th year Computer Science student, Aashish Harishchandre (harishch) a 3rd year Computer Science student, Kyle Rockwell (rockwe60) a 2nd year Computer Science student, and Saatvik Palli (pallisaa) a 3rd year Computer Science student, all students at Michigan State University. For our initial run, we decided to only implement the Feedforward Neural Network. We then, later, improved upon

our results using a Sequential Convolutional Neural Network.

We are trying two different models using two different methodologies to compare a simpler, more bare bones model Feedforward Neural Network, and a larger more complex Sequential Convolution Neural Network. We were inspired by a mutual love for poker and the realized lack of reliable card readers. The identification of playing cards is a fascinating area of current Machine Learning and Artificial Intelligence research. Having 16 discrete qualifiers, 4 suits and 12 faces, provides an interesting blend of object identification and image classification. At the center of our research is the labeled dataset of 8,029 labeled images of various playing cards with different colors, orientations, and lighting conditions. The benefit of our data set is not only the size but that it does not only contain the basic French Suited Standard English Style deck, but many cultural variations on design, pattern, and color.



Figure 1: Example cards from the dataset

This project has various applications: since all of our group members enjoy poker and other card games, having an effective model for identifying cards could prove useful for either personal projects or industry tools, such as an AI poker coach, automatic card counter for blackjack, or an accessibility aid for visually impaired players.

2 Related Works

Our team found two main existing papers on the subject. The first was “Playing Card Detection in Real Poker Scenes Using an Improved Efficient Model” by Qianmin Chen et al., published by IEEE in 2019 out of Qingdao, China. They achieved 91.5% precision and 93.3% recall using an EfficientNet variant optimized for detecting cards occupying just 0.4%–0.9% of the image area in cluttered poker scenes. We except from the outset that our models will not come close to this accuracy as they used a much simpler dataset.

The second was “Playing Cards Classification and Detection Using Sequential CNN Models Through Machine Learning Techniques Using Artificial Intelligence” by K. Mittal et al., published by IEEE in 2024. They employed a sequential CNN model optimized for card classification.

We also found two notable open source projects. The first was by Sanjit Dandapanthula¹, a Ph.D. student at Carnegie Mellon, that uses a Deep Convolutional Neural Network and a Feedforward Neural Network to classify the Fashion MNIST data set. We based our initial model off this Feed Forward Neural Network, updating and adjusting for our playing card dataset.

The second open source project we found was “Playing Card Classifier using Convolutional Neural Networks (CNNs)” by Hiroon Wijekoon³. This project demonstrated a 53-class card recognition using a 7,794-image

dataset, each image 224 x 224 pixels. Wijekoon’s architecture used two convolutional layers with max-pooling and ReLU. We plan to use this repository as a stepping stone for our eventual Sequential Convolutional Neural Network.

We initially planned to develop a Hybrid CNN architecture based of an EfficientNet, but felt that a simpler Sequential CNN would be much more equipped for the task and could classify accurately and efficiently. While we chose to use the much simpler Sequential CNN, it could be argued that we could have had better results sticking to our initial plan.

3 FNN Methodology

Our methodology for the first part of this project focuses on our feedforward neural network. This neural network can be seen in FeedforwardNN.ipynb in our repository. We used the pytorch library for our neural network. For training our neural network, we use cross entropy loss as our loss function, and stochastic gradient descent as our optimizer.

The groundwork for this implementation comes from Dandapanthula’s FNN¹, which we used as a baseline to ensure we had stable and versatile code for our neural network. We then adjusted Dandapanthula’s code to use our version of the model, and added various critical features such as saving and loading of created models during training.

Since we use a feedforward neural network, we first flatten our data into a vector, which we then pass through the following layers:

First, a linear layer reducing the vector from a list of 128x128 RGB pixels to 256 values. We then use a variation of the ReLU activation function provided by torch, RReLU, or the randomized leaky rectified linear unit function (Xu). This function is similar to the normal ReLU function, but values below 0 return a negative noisy value. This helps to prevent

¹ <https://github.com/sanjitdp/image-classification/tree/main>

³ [https://github.com/hiroonwijekoon/pytorch-cnn-](https://github.com/hiroonwijekoon/pytorch-cnn-playing-cards-classifier)

[playing-cards-classifier](https://github.com/hiroonwijekoon/pytorch-cnn-playing-cards-classifier)

overfitting, although it was not enough to fully eliminate it in this version of our model.

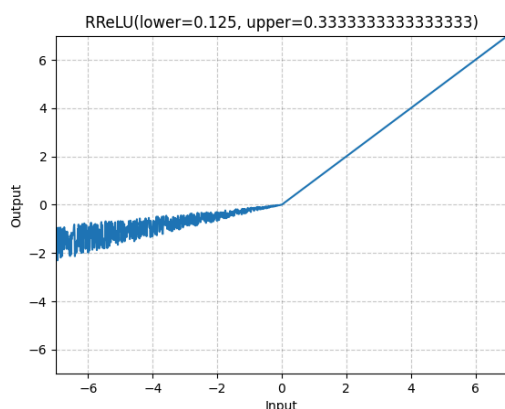


Fig 2: RReLU function (from pytorch.org)

Next, we use another linear layer from 256 values to 128 values, followed by the traditional ReLU activation function. We then use a linear layer from 128 to 100 values, followed by the same ReLU activation function.

Lastly, we use a final linear layer from 100 to 52 values, which we can find the maximum of to get the model's predicted classification of an image, as shown at the end of our code.

For training, we split our dataset into 90% training and 10% validation, using a set seed for consistent shuffling. We adjust each image in the dataset to be 128x128 pixels, to make processing easier and more consistent.

We trained the model for 1000 epochs, and have saved the resulting model as `fnn_cards_e999_acc60.pt` in our code for future use or validation.

4 FNN Experimental Results

Our initial model proved to be surprisingly effective at identifying cards, especially given the wide variety of designs within the dataset. The final accuracy of the model landed at 58.23%.

We used four metrics to measure the success of our model: accuracy, precision, recall, and F1. For our final results for our baseline Feedforward Neural Network, we had the following readings:

Accuracy = 58.229%

Precision = 58.684%

Recall = 58.823%

F1 Score = 57.749%

We believe the similarity between all the metrics is the result of the similarity of our dataset. If there is little variation between which classes cause misclassifications, we would expect accuracy to nearly match precision and that is what we saw. Recall and F1 score follow the same logic.

VALIDATION ACCURACY

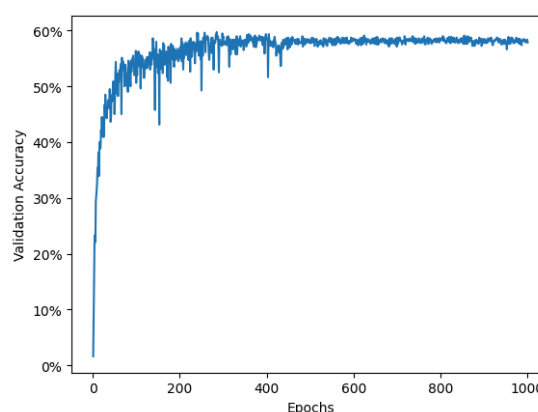


Fig 3: Graph of validation accuracy for FNN

As seen in the figure above, our model rapidly grew in accuracy before settling around 58% accuracy at roughly 400 epochs. Notably, the epochs after this were much more consistent compared to the noisy changes before it, likely because the model learned as much as it could from our dataset and there was no real benefit to further training, only increasing the risk of overfitting. It is also interesting to note that the model quickly approached near its peak accuracy, likely due to it being quite simple and our dataset being relatively small.

TRAINING LOSS

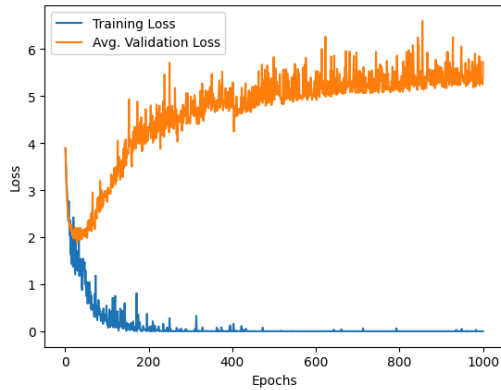


Fig 4: Graphs of loss testing and validation loss for the FNN

Figure illustrates the discrepancy between our model's training loss and validation loss. While training loss consistently decreased until nearly reaching 0, the validation loss decreased for about 20 epochs before growing even larger than the original value. This result indicates that our model is having an issue with overfitting, which we will have to watch for in our future CNN model. There are multiple avenues we can use to go about this, such as implementing neuron loss or using different activation functions to combat overfitting, and we will evaluate which methods work best for our final model.

5 Sequential CNN Methodology

Our methodology for Sequential Convolutional Neural Network based playing card classification model involves a layered architecture designed to extract and interpret hierarchical features from input images. The Sequential CNN Consists of 2 components; a convolutional network for feature extraction, and a fully connected classifier. For the first part, the model architecture consists of 2 convolutional layers, with ReLU activation functions, and 2-Dimensional max pooling to downsample the data.

The process begins with a series of convolutional layers that apply trainable filters to detect localized patterns such as edges, shapes, and symbols unique to each card's rank and suit. These are followed by pooling layers, which reduce the spatial dimensions of the feature

maps, enhancing translational invariance and reducing computational complexity. The resulting feature representations are then passed through fully connected layers, which integrate and interpret the learned spatial features for final classification.

A ReLU activation function in the output layer assigns probabilities to each of the 52 possible card classes. When training the model, we used various epoch counts as training cutoffs, due to the amount of compute available to us. Our final model was trained with 200 epochs.

The main reason we decided to run on less epochs was because we saw the accuracy of the FNN plateau at around 200-300 epochs while the validation loss continued to increase. By cutting off training at an earlier epoch, we believed we could prevent some of that overfitting issue we ran into before with our Feedforward Neural Network. Additionally, we ran into issues running our Sequential CNN on GPUs so were forced to run on CPUs which significantly increased training time. Using lower epochs assisted in this.

6 Sequential CNN Results

Our more advanced model showed a noticeable increase in both accuracy and efficiency, landing at a final accuracy of 64.09%.

We used the same four metrics to measure the success of our model: accuracy, precision, recall, and F1. For our final results for our baseline FNN model, we had the following readings:

Accuracy = 64.0898%

Precision = 64.5328%

Recall = 64.2596%

F1 Score = 63.3927%

The similarity of these results mimic the findings from our FNN. This adds further evidence to our theory that it is caused by the uniformity dataset where, for example, 4's

won't be any harder or easier to recognize than 5's.

VALIDATION ACCURACY

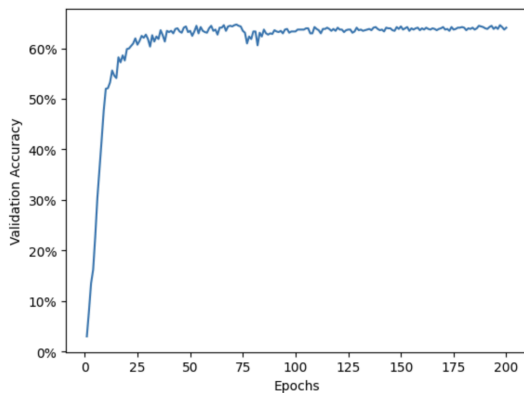


Fig. 5: Graph of validation accuracy for CNN

Figure 5 shows that the shape of the graph for validation accuracy for the Sequential CNN was very similar to that of our baseline FNN model. The key difference being that we found the Sequential CNN to be much more efficient, in the sense that it reached its peak and plateau of around 64%, at around 25-50 epochs, much sooner than the FNN did, at around 200-400 epochs. This provides further credence to the improvements of the Sequential CNN.

TRAINING LOSS

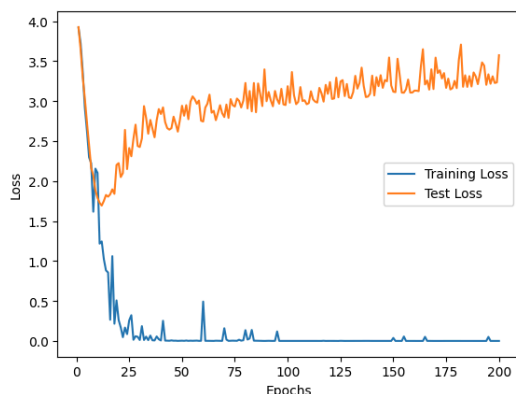


Fig. 6: Graphs of training and validation loss of the Sequential CNN

Figure 6 shows another improvement of the Sequential CNN over the FNN, while still displaying the same, although reduced, problem of overfitting. The training loss fell sharply and plateaued at around 25 epochs. It

also stopped at a loss of around 3 while the FNN plateaued around 5-6. This suggests that the CNN learned more efficiently but achieved lower training error but still shows a gap between training and validation loss, displaying the issue of residual overfitting.

7 Analysis

There were noticeable patterns between where the playing cards were successful and unsuccessful classified by our model.

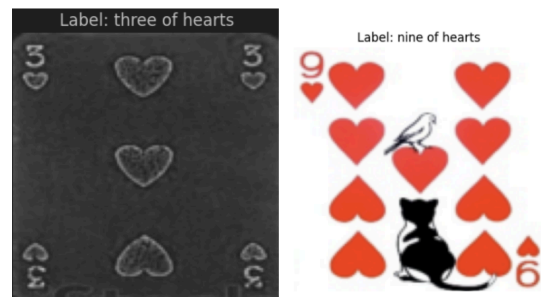


Fig. 7: Examples of correct classifications

Our model was very successful at recognizing standard images of cards, especially the standard french suited deck, that had no rotations, defects, or large amounts of card art. While we weren't able to place a solid metric on it, we rarely found misclassified cards that fit this criteria. On the other hand, when there was a significant rotation, defect in the data, or a large amount of card art, we noticed that our model was much more prone to misclassifications.



Fig 8. Example of incorrect classification due to rotated card

The first noticeable pattern we encountered when looking through the misclassifications of our model was that our model really struggled if the image was at all rotated or skewed. This implies that the model heavily relied on the suit rank pair that is visible in the upper left corner and bottom right corner of almost all the playing cards in our data set. When the card is rotated, that info is no longer in these corners so the model has that crucial piece of information translated.

In hindsight, if we would have provided more data augmentation with a random rotation of approximately 30 degrees, our model would have been much more robust and would have been better equipped at handling these discrepancies in our data. Using this technique could have additionally helped the overfitting of training data we experienced.

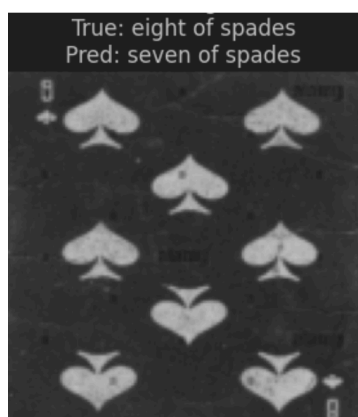


Fig. 9: Example of incorrect classification due to error in dataset

Another large issue we noticed when running our model was the overall defect of the dataset we used. On our initial run through of the dataset, it wasn't overtly obvious all the flaws it had that our model was able to pick up. There were many instances, such as in Figure 9, where there was no tangible way for even a human to classify which classification the card should fall under. Either issues with the rank, suit, or both, there were several cards that should not have been included in our training data at all. On further examination, it would have been very beneficial to the success of our model to manually go through and carefully and thoroughly examine each card to attempt to catch these cards at the outset. Both to prevent errors in the training of our model but

also to raise the artificially lowered accuracies caused by the to be expected misclassification of these cards.



Fig. 10: Example of incorrect classification due to card art

The pattern where our model struggled the most was when the card had a large amount of card art, as seen in Figure 10. These types of images took up approximately 35% - 55% of our entire dataset. When there was a small amount of art of the cards, it still fared fairly well as seen in Figure 7. But looking through our models' correct and incorrect classifications, there definitely seemed to be a strong relationship between the amount of card art and how likely it was to be misclassified.

In summary, looking through these findings, we noticed that we could have improved the accuracy of our model by finding a solution for these three problems. Notably, implementing more data augmentation and removing the faulty images from our dataset.

8 Future Works

To further improve our model, we could explore a parallel model approach, where two separate neural networks are trained simultaneously—one to predict the suit and one to predict the rank of the card. This design reflects the natural separation of the classification task into two distinct visual components, each with its own cues: suit symbols often appear in different positions and forms than rank indicators.

By allowing each model to specialize, we reduce the overall complexity of what each network needs to learn and potentially improve interpretability, accuracy, and precision. Additionally, this split-task approach can reduce class confusion (e.g., mistaking a heart for a diamond or a 5 for a 6), especially in cases where similar-looking cards introduce ambiguity in a single-label classifier. We believe this dual-model design, coupled with regularization techniques such as dropout and data augmentation (e.g., brightness and hue adjustments), will help mitigate overfitting and bring us closer to state-of-the-art accuracy benchmarks, even in the presence of our more complex and varied dataset.

The main issue any future works would need to tackle is the issue of overfitting. We believe this was the largest negative impact we had for our model. The tradeoff of running on larger epochs while overcoming overfitting, potentially by introducing early stopping techniques or other optimizers, would greatly improve the success of any future models.

10 Conclusion

In conclusion, our project involved training and evaluating two baseline models for playing card classification: a Feedforward Neural Network and a Sequential Convolutional Neural Network. The FNN achieved 58% accuracy but showed clear signs of overfitting, with training loss decreasing steadily while validation loss began to rise after only a few epochs. This model struggled to generalize due to its lack of spatial structure and reliance on dense layers. In contrast, the SCNN leveraged convolutional layers to better preserve spatial relationships in the input images, ultimately achieving a higher accuracy of 64%, while still being susceptible, although less so, to overfitting.

While this marks an improvement, both models fell short of the performance achieved by Mittal

et al. and Chen et al., whose CNN-based architectures reached accuracies as high as 91.5% on less complex datasets.

References

- K. Mittal, K. S. Gill, R. Chauhan, M. Sharma, G. Sunil. 2024. [Playing Cards Classification and Detection Using Sequential CNN Model Through Machine Learning Techniques Using Artificial Intelligence](#). 2024 *International Conference on E-mobility, Power Control, and Smart Systems (ICEMPS)*, Thiruvananthapuram, India, pp. 1-4
- Q. Chen, E. Rigall, X. Wang, H. Fan, J. Dong. 2019. [Poker Watcher: Playing Card Detection Based on EfficientDet and Sandglass Block](#). 2020 *11th International Conference on Awareness Science Technology (iCAST)*, Qingdao, China pp. 1-6
- Xu, B., et al. "Empirical evaluation of rectified activations in convolutional network," in arXiv preprint arXiv:1505.00853, 2015.
- Dandapanthula, Sanjit. [image-classification](#), *Github*, 1 Sep. 2022, <https://github.com/sanjitdp/image-classification/>
- Wijekoon, Hiroon. [Model Playing Card Classifier using Convolutional Neural Networks \(CNNs\)](#), *Github*, 17 Feb. 2024, <https://github.com/hiroonwijekoon/pytorch-cnn-playing-cards-classifier>